

Artifact Description

The artifact selected for this category is the same Android inventory app used in the first category from CS 360: Mobile Architecture and Programming. The original application, a simple inventory management app that stored items in an SQLite table and displayed them in a GridLayout, had no sorting, searching, or in-memory data organization of any kind. Items appeared in insertion order and every data change required a full database query. This application was chosen as the artifact for all three enhancement categories because it provides a perfect foundation for a scalable codebase with real algorithmic weaknesses that could have meaningful and well-noticed improvements.

Justification for Inclusion

I chose this artifact for the Algorithms and Data Structures category because the original implementation had no algorithmic design whatsoever. Items were merely loaded from the database in the order in which they were inserted, with no ability to sort, search, or efficiently look up by category. Every data change required a full database query with no in-memory data structure to organize items. This gave a clear baseline from which I could improve upon.

The enhancement introduced three algorithm features implemented in a dedicated class called SortSearchManager. The first is a HashMap-based category index that's built in at load time. When items are fetched from the database, they are organized into a HashMap using a String and a List of MediaItems, keyed by category name. This allows the application to retrieve all items in a given category in $O(1)$ average case time. The original scan had a case time of $O(n)$ using a linear scan. The index is rebuilt whenever the item list changes, ensuring it will always reflect current data. The second feature is a merge sort implementation that powers the sort menu in each category tab. Merge sort was chosen for its stable sorting algorithm. This means that if items compare as equal on the chosen sort field, they will retain their previous relative order. This matters because a user sorting by rating would expect two four-star movies to remain in their previously sorted title order rather than being swapped arbitrarily. A more unstable sort, Quicksort, would not give this guarantee. The Merge sort runs in $O(n \log n)$ time in all cases and uses $O(n)$ auxiliary space for the merge step. The implementation provides support for sorting books, movies, and video games by title, year, or rating in either ascending or descending order (i.e., ratings from high to low or vice versa), giving users six sort combinations through a popup menu. The third feature is a binary search function used for title lookups when the list is sorted alphabetically by title. Binary search runs in $O(\log n)$ time by halving the search space on every step, compared to an $O(n)$ linear scan that would be required. Since binary search requires a

sorted list to function correctly, the application automatically falls back to a linear search when the user has sorted by year or rating. This provides an example of the understanding of preconditions that algorithms require and the importance of choosing the right algorithm for the current state of the data.

Course Outcomes

The planned enhancement for Category Two targeted course outcome three: design and evaluate computing solutions that solve a given problem using algorithmic principles and computer science practices and standards appropriate to its solution, while managing the trade-offs involved in design choices. This outcome can easily be seen in the decision to choose Merge sorting over any other sorting options. The trade-off of requiring sorted lists provides Merge sorting with optimal search speed when it comes to scaling up the database. A simple linear search would not be able to scale properly, especially with three different categories of media items.

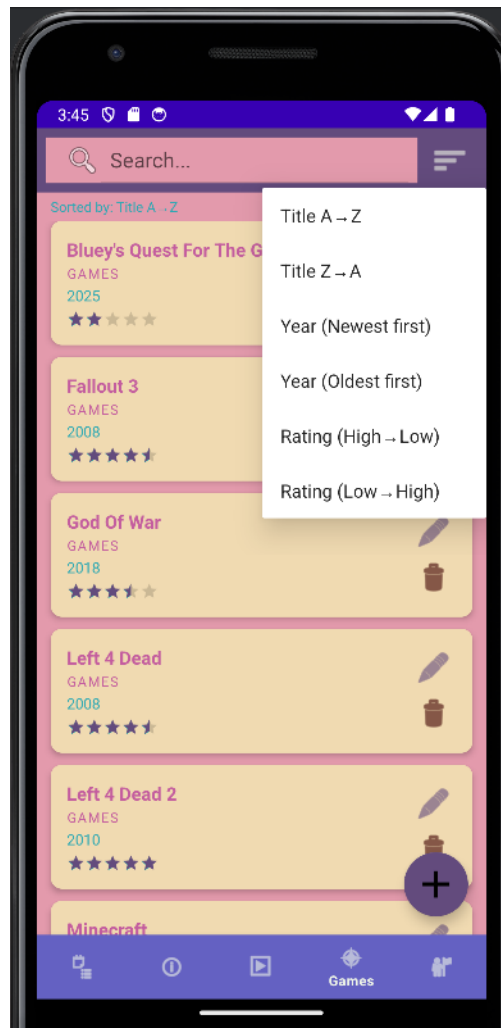
Process Reflection

One of the most significant things I learned during this enhancement was the difference between theoretical algorithmic properties and the actual implementation and constraints of the algorithms. For instance, binary search is simple to implement by itself but connecting it to the rest of the application revealed a dependency that had to be managed: binary search only works correctly on a list sorted by the same field being searched. Implementing the smart search that checks the current sort field before deciding which search algorithm to use was a result of this dependency. This produced a more robust code.

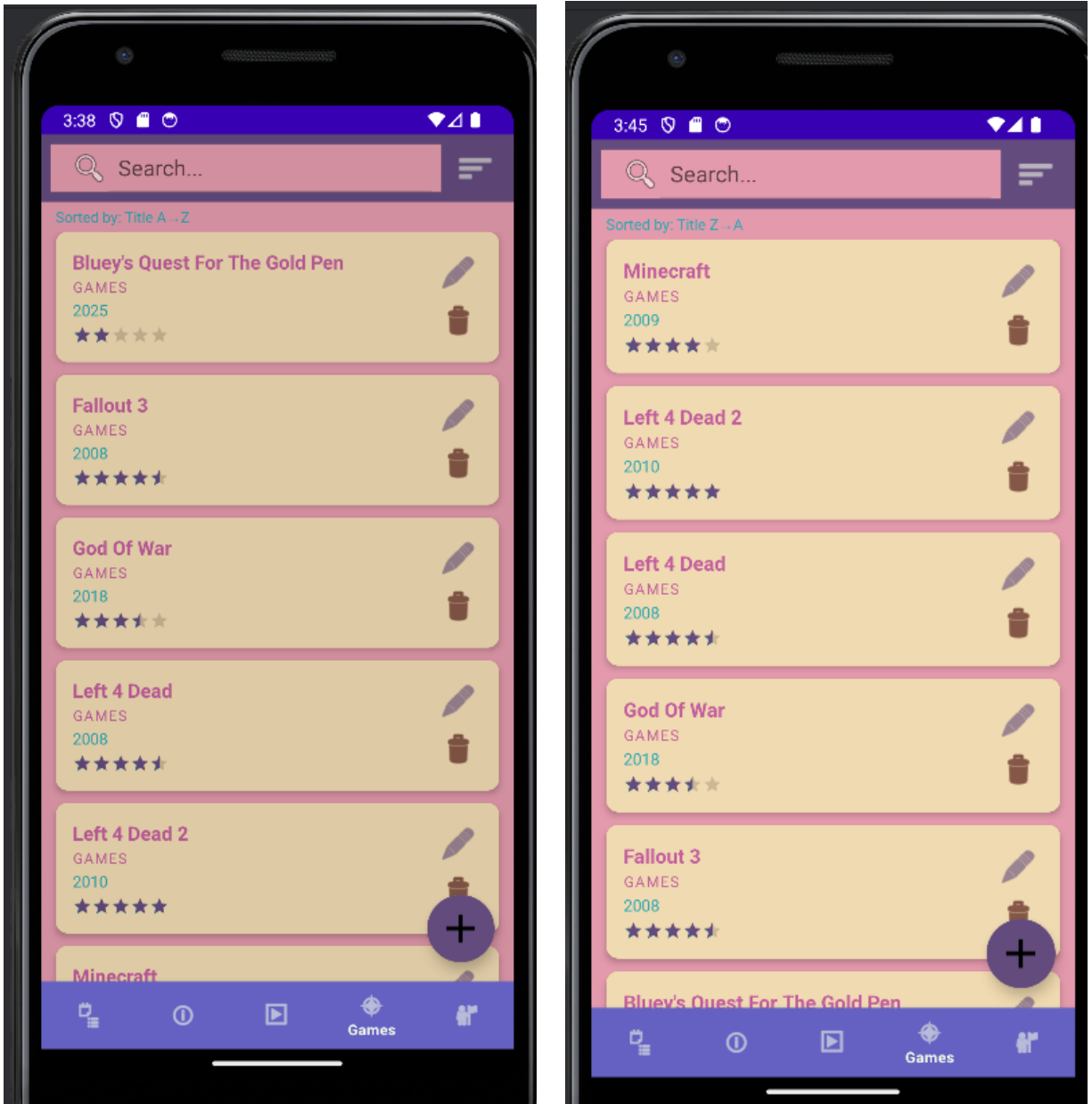
Another important lesson came from implementing the merge sort. An error I introduced in the merge function made it use the wrong list's size in the final loop, which caused the sort to silently produce incorrect results without throwing any exception. This was difficult to detect and took thorough code examination to determine the cause. Although the solution was simple, it was a great reminder to be diligent with even the smallest portions of code.

Overall, the category two enhancement has produced a more capable and user-friendly designed application. The process of implementing it deepened my understanding of algorithms and data structure concepts that I can apply to current and future software development problems.

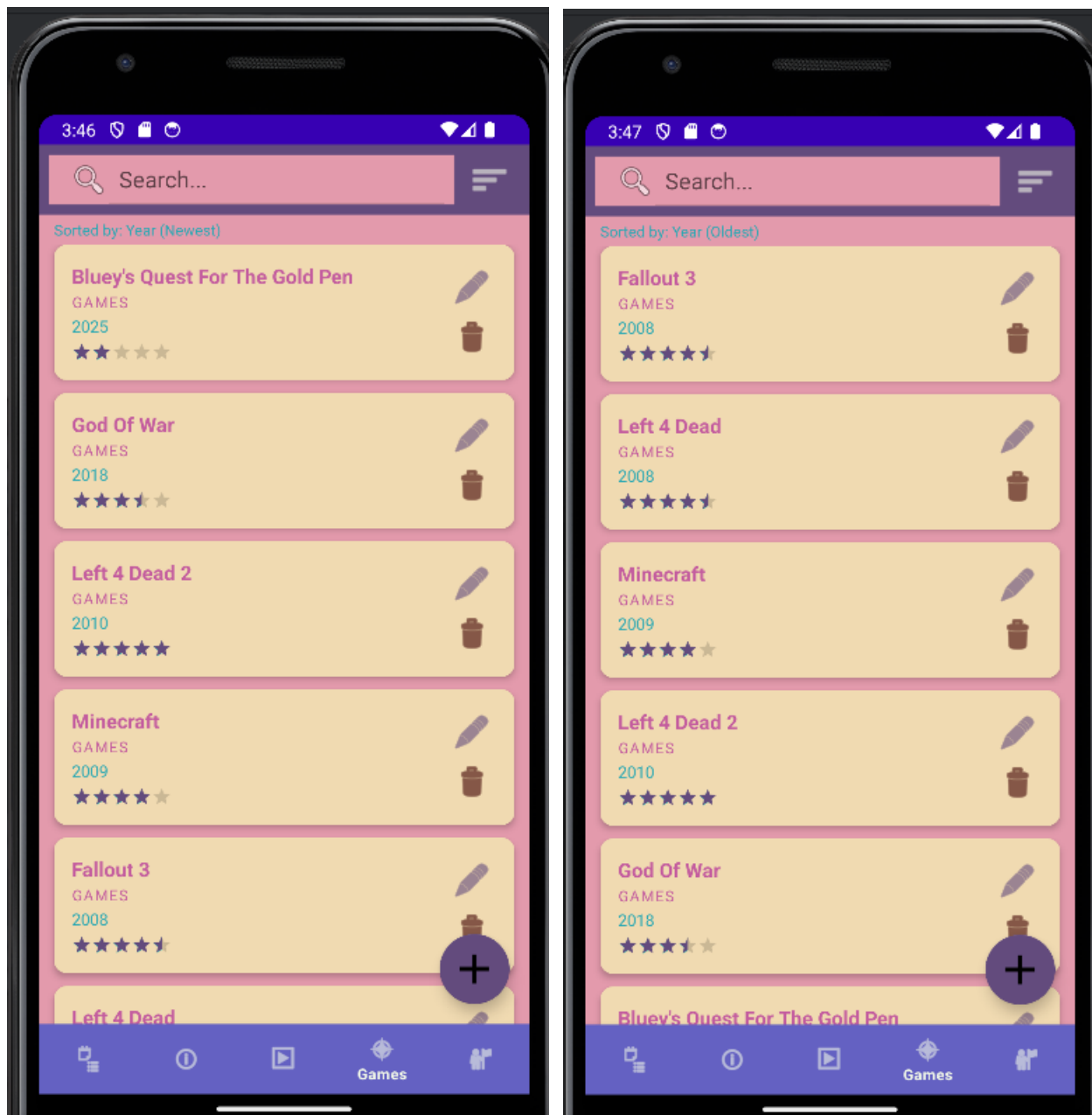
Photographic Evidence



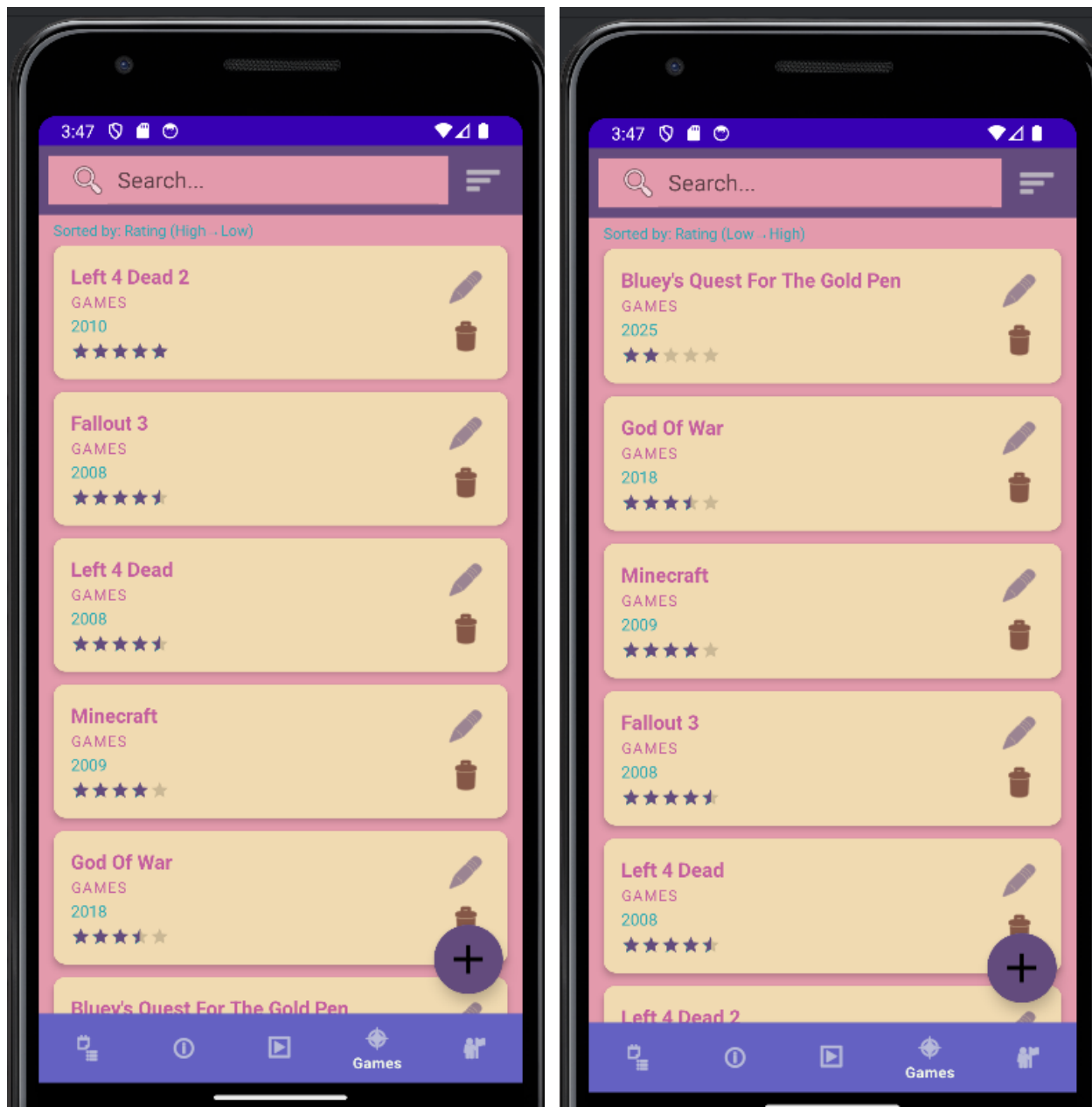
Above is an example of the sort menu used to sort books, movies, or video games using titles, year released, or rating, in either ascending or descending order.



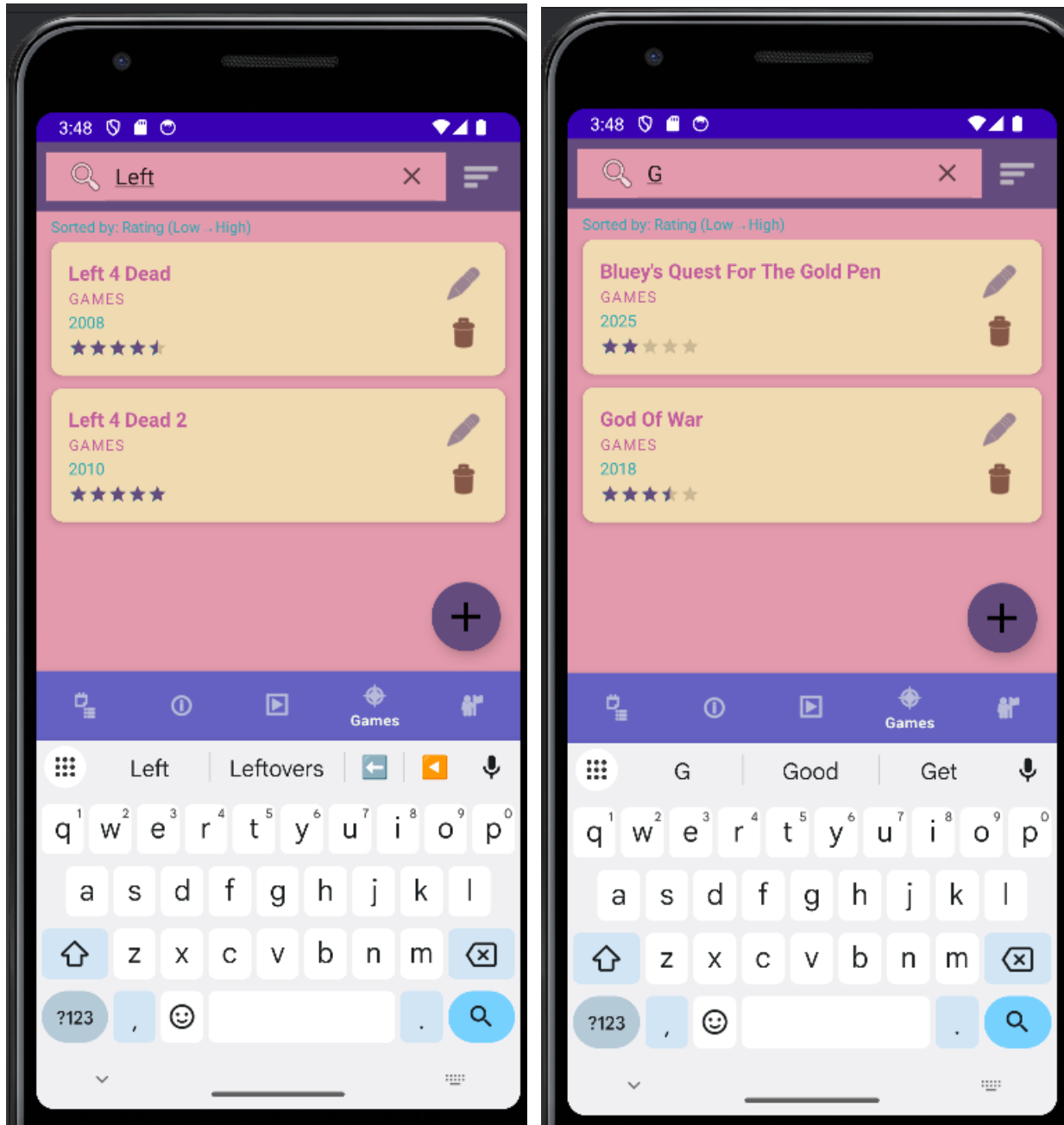
Above is a demonstration of sorting the games tab by title. On the left, it is sorted from A to Z, and on the right, Z to A.



Above is a demonstration of sorting the games tab by the year they were released. On the left, it is sorted from newest to oldest, and on the right, oldest to newest.



Above is a demonstration of sorting the games tab by rating. On the left, it is sorted from highest to lowest, and on the right, lowest to highest.



And lastly, the two images above provide an example of the search function. On the left, I have searched for the word “Left” and have been provided with all games that include the word “left.” On the right, I simply searched “G” and was provided all games that included a “G” anywhere in their title.